

REST Assured API Testing

Complete Guide for QA Testers

Java • Groovy Gradle • Given/When/Then • POJO • Builder Pattern

Version 1.0 | For Beginner QA Engineers

1. Introduction

REST Assured is a Java library that simplifies testing RESTful APIs. Combined with the Given/When/Then (BDD) pattern, it makes API tests readable even for beginners. This guide walks you through a complete, professional-grade test framework from scratch.

What You Will Learn

- Project structure and Groovy Gradle setup
- Base class for reusability
- POJO classes with @JsonProperty and Builder pattern
- Writing Given / When / Then API tests
- Handling complex JSON with lists and nested objects

2. Project Structure

A well-organised project separates concerns. Here is the recommended folder layout:

my-api-tests/		
—	build.gradle	← Groovy Gradle build file
—	src/	
	└─ test/	
	└─ java/	
	└─ com/	
	└─ mycompany/	
	└─ base/	
	└─ BaseTest.java	← Base class (shared setup)
	└─ pojo/	
	└─ BookList.java	← POJO for top-level list
	└─ Book.java	← POJO for individual book
	└─ Author.java	← POJO for nested author object
	└─ ApiResponse.java	← POJO for API wrapper response
	└─ tests/	
	└─ GetBooksTest.java	← GET tests
	└─ CreateBookTest.java	← POST tests
	└─ UpdateBookTest.java	← PUT tests

 **Tip:** Never put test logic inside your base class. The base class only handles setup and configuration. Test classes extend BaseTest and contain the actual test methods.

3. Groovy Gradle Setup (build.gradle)

Create this file at the root of your project. It pulls in all required libraries.

plugins {
id 'java'
}
group = 'com.mycompany'
version = '1.0'
sourceCompatibility = '11'
repositories {

```
mavenCentral()
}

dependencies {
    // REST Assured – the core API testing library
    testImplementation 'io.rest-assured:rest-assured:5.4.0'

    // Jackson – converts JSON to Java objects (POJO)
    testImplementation 'com.fasterxml.jackson.core:jackson-databind:2.16.1'

    // TestNG – test runner (provides @Test, @BeforeClass, etc.)
    testImplementation 'org.testng:testng:7.9.0'

    // Hamcrest – for assertThat() matchers
    testImplementation 'org.hamcrest:hamcrest:2.2'
}

test {
    useTestNG() // tell Gradle to run TestNG (not JUnit)
}
```

 **Note:** Run 'gradle test' from the project root to execute all tests. The first run downloads all dependencies automatically.

4. POJO Classes

POJO stands for Plain Old Java Object. These classes represent the JSON data structure your API sends or receives. Each field is annotated with `@JsonProperty` to map the JSON key to the Java field name.

The API we test returns this JSON structure:

```
{
  "status": "success",
  "totalCount": 2,
  "bookList": [
    {
      "bookName": "Clean Code",
      "bookCount": 5,
      "author": {
        "authorName": "Robert C. Martin",
```

```
    "authorId": 101
  },
  "tags": ["programming", "best-practices"]
},
{
  "bookName": "The Pragmatic Programmer",
  "bookCount": 3,
  "author": {
    "authorName": "David Thomas",
    "authorId": 102
  },
  "tags": ["programming", "career"]
}
]
```

4.1 Author.java (nested object)

This POJO maps to the "author" object nested inside each book.

```
package com.mycompany.pojo;

import com.fasterxml.jackson.annotation.JsonProperty;

public class Author {

    @JsonProperty("authorName")
    public String authorName;

    @JsonProperty("authorId")
    public int authorId;

    // — Required: no-arg constructor for Jackson deserialization —
    public Author() {}

    // — Private constructor used only by Builder —
    private Author(Builder builder) {
        this.authorName = builder.authorName;
        this.authorId = builder.authorId;
    }

    // — Builder Pattern —
```

```
public static class Builder {  
    private String authorName;  
    private int    authorId;  
  
    public Builder authorName(String authorName) {  
        this.authorName = authorName;  
        return this;  
    }  
  
    public Builder authorId(int authorId) {  
        this.authorId = authorId;  
        return this;  
    }  
  
    public Author build() {  
        return new Author(this);  
    }  
}
```

4.2 Book.java (list item with nested object + list)

Each element inside the "bookList" array maps to a `Book` object. It contains a nested `Author` object and a `List<String>` for tags.

```
package com.mycompany.pojo;  
  
import com.fasterxml.jackson.annotation.JsonProperty;  
import java.util.List;  
  
public class Book {  
  
    @JsonProperty("bookName")  
    public String bookName;  
  
    @JsonProperty("bookCount")  
    public int bookCount;  
  
    @JsonProperty("author")  
    public Author author;           // ← nested object  
  
    @JsonProperty("tags")
```

```
public List<String> tags;    // ← list of strings
```

```
// — No-arg constructor for Jackson —
```

```
public Book() {}
```

```
// — Private constructor used by Builder —
```

```
private Book(Builder builder) {  
    this.bookName = builder.bookName;  
    this.bookCount = builder.bookCount;  
    this.author = builder.author;  
    this.tags = builder.tags;  
}
```

```
// — Builder Pattern —————
```

```
public static class Builder {
```

```
    private String    bookName;  
    private int       bookCount;  
    private Author    author;  
    private List<String> tags;
```

```
    public Builder bookName(String bookName) {  
        this.bookName = bookName;  
        return this;  
    }
```

```
    public Builder bookCount(int bookCount) {  
        this.bookCount = bookCount;  
        return this;  
    }
```

```
    public Builder author(Author author) {  
        this.author = author;  
        return this;  
    }
```

```
    public Builder tags(List<String> tags) {  
        this.tags = tags;  
        return this;  
    }
```

```
    public Book build() {  
        return new Book(this);  
    }
```

```
}  
}
```

4.3 BookList.java (top-level list wrapper)

This is the root-level POJO — exactly matching the pattern you specified. It wraps the list of books and additional metadata from the API response.

```
package com.mycompany.pojo;  
  
import com.fasterxml.jackson.annotation.JsonProperty;  
import java.util.List;  
  
public class BookList {  
  
    @JsonProperty("bookName")  
    public String bookName;  
  
    @JsonProperty("bookCount")  
    public int bookCount;  
  
    // No-arg constructor – required for Jackson to deserialize JSON  
    public BookList() {}  
  
}
```

4.4 ApiResponse.java (full API wrapper)

The top-level wrapper that maps the entire API response including `status`, `totalCount`, and the `bookList` array.

```
package com.mycompany.pojo;  
  
import com.fasterxml.jackson.annotation.JsonProperty;  
import java.util.List;  
  
public class ApiResponse {  
  
    @JsonProperty("status")  
    public String status;  
  
}
```

```
@JsonProperty("totalCount")
public int totalCount;

@JsonProperty("bookList")
public List<Book> bookList;    // ← list of Book objects

// — No-arg constructor for Jackson —
public ApiResponse() {}

// — Private constructor used by Builder —
private ApiResponse(Builder builder) {
    this.status      = builder.status;
    this.totalCount = builder.totalCount;
    this.bookList    = builder.bookList;
}

// — Builder Pattern —————
public static class Builder {
    private String      status;
    private int         totalCount;
    private List<Book>  bookList;

    public Builder status(String status) {
        this.status = status;
        return this;
    }

    public Builder totalCount(int totalCount) {
        this.totalCount = totalCount;
        return this;
    }

    public Builder bookList(List<Book> bookList) {
        this.bookList = bookList;
        return this;
    }

    public ApiResponse build() {
        return new ApiResponse(this);
    }
}
```


 **Builder Pattern Explained:** Instead of calling `new Book("Clean Code", 5, author, tags)` with confusing positional args, the builder lets you write: `new Book.Builder().bookName("Clean Code").bookCount(5).author(author).build()` — much clearer and maintainable.

5. Base Test Class

The base class contains everything shared across all tests: base URL, authentication headers, timeout settings, and common setup/teardown logic. All test classes extend this class — they never copy-paste this setup code.

```
package com.mycompany.base;

import io.restassured.RestAssured;
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.builder.ResponseSpecBuilder;
import io.restassured.filter.log.RequestLoggingFilter;
import io.restassured.filter.log.ResponseLoggingFilter;
import io.restassured.http.ContentType;
import io.restassured.specification.RequestSpecification;
import io.restassured.specification.ResponseSpecification;
import org.testng.annotations.BeforeClass;

public class BaseTest {

    // — Shared specifications (reused by all tests) —————
    protected RequestSpecification requestSpec;
    protected ResponseSpecification responseSpec;

    // — Constants – change these to switch environments —————
    private static final String BASE_URL = "https://api.example.com";
    private static final String API_KEY = "your-api-key-here";
    private static final int TIMEOUT_MS = 10000;

    @BeforeClass
    public void setUp() {

        // Build request spec – applied to EVERY request automatically
        requestSpec = new RequestSpecBuilder()
            .setBaseUrl(BASE_URL) // base URL for all requests
            .setContentType(ContentType.JSON) // Content-Type: application/json
            .addHeader("X-API-Key", API_KEY) // auth header on every request
```

```
.addFilter(new RequestLoggingFilter()) // log requests in console
.addFilter(new ResponseLoggingFilter()) // log responses in console
.build();

// Build response spec — common assertions for EVERY response
responseSpec = new ResponseSpecBuilder()
    .expectContentType(ContentType.JSON) // always expect JSON back
    .build();

// Global timeout — fail if server takes longer than 10 seconds
RestAssured.config = RestAssured.config()
    .httpClient(io.restassured.config.HttpClientConfig
        .httpClientConfig()
        .setParam("http.connection.timeout", TIMEOUT_MS)
        .setParam("http.socket.timeout", TIMEOUT_MS));
}
```

 **Key Concept:** RequestSpecification is like a template. Once built in BaseTest, every test class inherits it and simply calls `given().spec(requestSpec)` — no need to repeat headers, URL, or auth in each test.

6. Test Classes — Given / When / Then

Every test follows three clear steps: **Given** (setup — what you send), **When** (action — the HTTP call), and **Then** (assertions — what you check).

6.1 GetBooksTest.java — GET Request

```
package com.mycompany.tests;

import com.mycompany.base.BaseTest;
import com.mycompany.pojo.ApiResponse;
import com.mycompany.pojo.Book;
import io.restassured.response.Response;
import org.testng.annotations.Test;
import org.testng.Assert;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;
```

```
public class GetBooksTest extends BaseTest {    // ← extends BaseTest

    // — Test 1: Verify status code and status field —————
    @Test
    public void testGetAllBooks_Returns200() {

        // GIVEN — set up the request
        given()

            .spec(requestSpec)                // use shared base config

        // WHEN — make the HTTP call
        .when()

            .get("/books")                    // GET /books

        // THEN — verify the response
        .then()

            .spec(responseSpec)              // check shared response rules
            .statusCode(200)                 // must be HTTP 200
            .body("status", equalTo("success")); // status field check
    }

    // — Test 2: Deserialize response into POJO and verify data —
    @Test
    public void testGetAllBooks_ListIsNotEmpty() {

        // GIVEN
        Response response =
            given()

                .spec(requestSpec)

        // WHEN
        .when()

            .get("/books")

        // THEN
        .then()

            .spec(responseSpec)
            .statusCode(200)
            .extract().response();           // extract raw response

        // Deserialize JSON → ApiResponse POJO
        ApiResponse apiResponse = response.as(ApiResponse.class);
```

```
// Assert on the POJO fields
Assert.assertEquals(apiResponse.status, "success");
Assert.assertNotNull(apiResponse.bookList, "bookList should not be null");
Assert.assertTrue(apiResponse.bookList.size() > 0, "bookList should have items");

// Loop and verify each book
for (Book book : apiResponse.bookList) {
    Assert.assertNotNull(book.bookName, "bookName must not be null");
    Assert.assertNotNull(book.author, "author must not be null");
    Assert.assertTrue(book.bookCount >= 0, "bookCount must be >= 0");
}
}

// — Test 3: Get a specific book by ID —————
@Test
public void testGetBookById_ReturnsCorrectBook() {

    int bookId = 1;

    // GIVEN
    given()
        .spec(requestSpec)
        .pathParam("id", bookId) // replace {id} in URL

    // WHEN
    .when()
        .get("/books/{id}")

    // THEN
    .then()
        .spec(responseSpec)
        .statusCode(200)
        .body("bookList[0].bookName", not(emptyOrNullString()))
        .body("bookList[0].bookCount", greaterThanOrEqualTo(0))
        .body("bookList[0].author.authorName", not(emptyOrNullString()));
}
}
```

6.2 CreateBookTest.java — POST Request with Builder

```
package com.mycompany.tests;
```

```
import com.mycompany.base.BaseTest;
import com.mycompany.pojo.Author;
import com.mycompany.pojo.Book;
import com.mycompany.pojo.ApiResponse;
import io.restassured.response.Response;
import org.testng.annotations.Test;
import org.testng.Assert;

import java.util.Arrays;
import java.util.List;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;

public class CreateBookTest extends BaseTest {    // ← extends BaseTest

    // — Helper: build an Author using Builder pattern —————
    private Author buildAuthor(String name, int id) {
        return new Author.Builder()
            .authorName(name)
            .authorId(id)
            .build();
    }

    // — Helper: build a Book using Builder pattern —————
    private Book buildBook(String name, int count, Author author, List<String> tags) {
        return new Book.Builder()
            .bookName(name)
            .bookCount(count)
            .author(author)
            .tags(tags)
            .build();
    }

    // — Test 1: Create a single book —————
    @Test
    public void testCreateBook_Returns201() {

        // Build request body using Builder pattern
        Author author = buildAuthor("Robert C. Martin", 101);
        Book book = buildBook(
            "Clean Code",
```

```
        5,
        author,
        Arrays.asList("programming", "best-practices")
    );

    // GIVEN
    given()
        .spec(requestSpec)
        .body(book) // serialize POJO → JSON automatically

    // WHEN
    .when()
        .post("/books") // POST /books

    // THEN
    .then()
        .spec(responseSpec)
        .statusCode(201) // expect 201 Created
        .body("status", equalTo("success"))
        .body("bookList[0].bookName", equalTo("Clean Code"))
        .body("bookList[0].author.authorName", equalTo("Robert C. Martin"));
    }

    // — Test 2: Create multiple books and verify the list —————
    @Test
    public void testCreateMultipleBooks_VerifyList() {

        Author author1 = buildAuthor("Robert C. Martin", 101);
        Author author2 = buildAuthor("David Thomas", 102);

        List<Book> books = Arrays.asList(
            buildBook("Clean Code", 5, author1,
                Arrays.asList("programming")),
            buildBook("The Pragmatic Programmer", 3, author2, Arrays.asList("career"))
        );

        // GIVEN - send a list of books
        Response response =
            given()
                .spec(requestSpec)
                .body(books)

        // WHEN
```

```
        .when()
            .post("/books/batch")

        // THEN
        .then()
            .spec(responseSpec)
            .statusCode(201)
            .extract().response();

        // Deserialize and assert
        ApiResponse apiResponse = response.as(ApiResponse.class);
        Assert.assertEquals(apiResponse.bookList.size(), 2);
        Assert.assertEquals(apiResponse.bookList.get(0).bookName, "Clean Code");
        Assert.assertEquals(apiResponse.bookList.get(1).author.authorName, "David Thomas");
    }
}
```

6.3 UpdateBookTest.java — PUT Request

```
package com.mycompany.tests;

import com.mycompany.base.BaseTest;
import com.mycompany.pojo.Author;
import com.mycompany.pojo.Book;
import org.testng.annotations.Test;

import java.util.Arrays;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;

public class UpdateBookTest extends BaseTest {    // ← extends BaseTest

    @Test
    public void testUpdateBook_Returns200() {

        int bookId = 1;

        // Build updated data using Builder pattern
        Author updatedAuthor = new Author.Builder()
            .authorName("Robert C. Martin")
            .authorId(101)
```

```
        .build();

Book updatedBook = new Book.Builder()
    .bookName("Clean Code – 2nd Edition")
    .bookCount(10)
    .author(updatedAuthor)
    .tags(Arrays.asList("programming", "updated"))
    .build();

// GIVEN
given()
    .spec(requestSpec)
    .pathParam("id", bookId)
    .body(updatedBook)

// WHEN
.when()
    .put("/books/{id}")

// THEN
.then()
    .spec(responseSpec)
    .statusCode(200)
    .body("bookList[0].bookName", equalTo("Clean Code – 2nd Edition"))
    .body("bookList[0].bookCount", equalTo(10));
}

@Test
public void testUpdateBook_InvalidId_Returns404() {

    Book book = new Book.Builder()
        .bookName("Nonexistent Book")
        .bookCount(1)
        .build();

    // GIVEN
    given()
        .spec(requestSpec)
        .pathParam("id", 99999)    // ID that does not exist
        .body(book)

    // WHEN
    .when()
```



```
        .put("/books/{id}")

// THEN
        .then()
            .statusCode(404);           // expect 404 Not Found
    }
}
```

7. Query Parameters & Path Parameters

APIs often accept filters via query parameters. Here is how to add them cleanly:

```
// Query parameters - appended to URL: /books?category=fiction&limit=10
given()
    .spec(requestSpec)
    .queryParams("category", "fiction")
    .queryParams("limit", 10)
    .when()
        .get("/books")
    .then()
        .statusCode(200);

// Path parameters - replace {placeholders} in URL: /books/42
given()
    .spec(requestSpec)
    .pathParam("id", 42)
    .when()
        .get("/books/{id}")
    .then()
        .statusCode(200);

// Multiple query params at once using a Map
Map<String, Object> params = new HashMap<>();
params.put("category", "fiction");
params.put("limit", 10);
params.put("page", 1);

given()
    .spec(requestSpec)
    .queryParams(params)
    .when()
```

<code>.get("/books")</code>
<code>.then()</code>
<code>.statusCode(200);</code>

8. Common Assertion Matchers (Cheat Sheet)

These matchers go inside `.then().body("jsonPath", matcher)`. They come from the Hamcrest library.

Matcher	What it checks
<code>equalTo("value")</code>	Exact match (string, int, boolean)
<code>not(equalTo("value"))</code>	Not equal to
<code>greaterThan(5)</code>	Number is > 5
<code>greaterThanOrEqualTo(0)</code>	Number is >= 0
<code>lessThan(100)</code>	Number is < 100
<code>hasSize(3)</code>	List has exactly 3 elements
<code>hasItem("value")</code>	List contains this item
<code>not(emptyOrNullString())</code>	String is not blank or null
<code>notNullValue()</code>	Field is not null
<code>containsString("part")</code>	String contains substring
<code>startsWith("prefix")</code>	String starts with prefix
<code>everyItem(notNullValue())</code>	Every item in list is not null

9. JsonPath — Navigating Complex JSON

JsonPath is like XPath but for JSON. Use it in `.body("path", matcher)` to pinpoint any value.

JsonPath Expression	What it selects
<code>status</code>	Top-level field

JsonPath Expression	What it selects
<code>bookList[0].bookName</code>	bookName of first book in list
<code>bookList[1].author.authorName</code>	authorName inside second book's author
<code>bookList.size()</code>	Total number of books in list
<code>bookList[0].tags[0]</code>	First tag of first book
<code>bookList.bookName</code>	List of all bookName values
<code>bookList.findAll{it.bookCount>2}</code>	Books where bookCount > 2

```
// Examples in tests
.body("bookList[0].bookName", equalTo("Clean Code"))
.body("bookList[0].author.authorId", equalTo(101))
.body("bookList[0].tags", hasItem("programming"))
.body("bookList.size()", equalTo(2))
.body("bookList.bookName", hasItems("Clean Code", "The Pragmatic Programmer"))
.body("bookList.findAll{it.bookCount>2}.size()", equalTo(1))
```

10. Extracting Values for Reuse Across Tests

Sometimes you need a value from one response to use in the next request (e.g., create a book, get its ID, then delete it).

```
// Extract a single value from JSON
String bookName = given()
    .spec(requestSpec)
    .when()
        .get("/books/1")
    .then()
        .statusCode(200)
        .extract().path("bookList[0].bookName");

System.out.println("Got book: " + bookName);

// Extract entire response and deserialize
Response response = given()
    .spec(requestSpec)
```

```
.when()
    .get("/books")
.then()
    .statusCode(200)
    .extract().response();

ApiResponse data = response.as(ApiResponse.class);
int firstBookCount = data.bookList.get(0).bookCount;

// Use extracted value in the next request
given()
    .spec(requestSpec)
    .queryParams("minCount", firstBookCount)
.when()
    .get("/books")
.then()
    .statusCode(200);
```

11. DELETE Request

```
package com.mycompany.tests;

import com.mycompany.base.BaseTest;
import org.testng.annotations.Test;

import static io.restassured.RestAssured.given;

public class DeleteBookTest extends BaseTest {

    @Test
    public void testDeleteBook_Returns204() {

        int bookId = 1;

        // GIVEN
        given()
            .spec(requestSpec)
            .pathParam("id", bookId)

        // WHEN
        .when()
```

```
        .delete("/books/{id}")

// THEN
        .then()

        .statusCode(204);    // 204 No Content = deleted successfully
    }

@Test
public void testDeleteBook_VerifyGoneAfterDeletion() {

    int bookId = 2;

    // Step 1: Delete the book
    given().spec(requestSpec).pathParam("id", bookId)
        .when().delete("/books/{id}")
        .then().statusCode(204);

    // Step 2: Verify it no longer exists
    given().spec(requestSpec).pathParam("id", bookId)
        .when().get("/books/{id}")
        .then().statusCode(404);
    }
}
```

12. Running Your Tests

Command	What it does
gradle test	Run all tests
gradle test --tests "/*.GetBooksTest"	Run one test class
gradle test --info	Verbose output
gradle clean test	Clean build then test

After running, find the HTML report at: build/reports/tests/test/index.html

13. Quick Reference Summary

Concept	Rule
@JsonProperty	Maps JSON key → Java field. Always required on POJO fields.
No-arg constructor	Required in every POJO. Jackson uses it to create objects.
Builder pattern	Use Builder.build() to create request body objects cleanly.
BaseTest	Contains shared setup. Never put test methods here.
extends BaseTest	Every test class must extend BaseTest to get requestSpec.
given()	Starts a request. Configure headers, body, params here.
when()	Executes the HTTP method (get, post, put, delete).
then()	Asserts on the response (status code, body, headers).
.spec(requestSpec)	Applies the shared base config to every request.
.as(Class.class)	Deserializes JSON response body into a POJO.
.extract().path()	Pulls a single value out of JSON for reuse.

 **You are ready!** You now have everything to write professional API tests with REST Assured. Start with GetBooksTest, run it, read the console logs, then gradually write POST and PUT tests. Ask questions, read the logs — they tell you exactly what went wrong.